

Speaker Hub — Security & Site Review

Automated review of speaker-hub.com · repo `mickhatzikostas220-design/speech-analyzer2` · 10 July 2026

Solid

PROD SECURITY
POSTURE

~15

UNMERGED SECURITY
PRS

1

NEW BUG FOUND

0

COMMITTED SECRETS

The short version

The site is in good security shape, and the reason is worth stating plainly: nearly every vulnerability a fresh audit turns up has **already been found and fixed** in a previous run — the fix is just sitting in an unmerged draft pull request.

Three independent audits this run (API-route auth, Supabase RLS, secrets/OAuth) covered the whole attack surface. They surfaced the storage-bucket exposure, the SSRF holes, editor IDOR, and rate-limit gaps you'd expect — and for each one there is *already an open PR that closes it*. Production itself was previously verified clean (all tables have RLS; the dangerous storage policy is already absent per `PRODUCTION_CHECKLIST.md`).

So the bottleneck is not finding bugs. It's merging the fixes. Roughly 15 draft "security hardening" PRs have accumulated (branches `compassionate-brahmagupta-*` are prior runs of this same routine), and none are merged. Each new run adds another. The single most useful thing right now is to triage that queue — merge one canonical hardening PR, close the redundant ones — rather than generate a 16th.

One finding does appear **NEW** and not covered by any open PR: a **billing-column protection trigger that is silently broken** and will block real Stripe/admin plan changes the moment paywalls go live. I've written a single run-once migration that fixes it (details below).

What this run changed

Added `supabase/hardening_2026_07.sql` — one idempotent, run-once script that closes the entire SQL-layer posture in a single place instead of across several PRs. Safe to run on already-patched production (every statement is `drop ... if exists / create or replace`). It:

- Drops the over-broad `"Service reads speeches"` storage policy (cross-tenant file read).
- Scopes the `feedback_points / engagement_timeline` INSERT policies to the owning analysis (were with `check (true)`).
- **Fixes the billing trigger role check** — the new finding (§ Billing trigger).
- Pins `search_path` on both `SECURITY DEFINER` functions (Supabase advisory).

I deliberately did **not** re-implement the SSRF guards, editor-IDOR path checks, or rate-limit additions in code — those already exist in open PRs (mapped below), and duplicating them would just deepen the same backlog this report is flagging. I also did not make speculative "improvements" to the live site: the project's own scope rule is to stay in your lane, and broad autonomous edits to a production app aren't a safe call to make unattended.

Every recurring finding already has an open PR

This is the core of it. The audits are re-discovering work that's done:

FINDING	SEVERITY	ALREADY FIXED IN
Storage bucket readable cross-tenant ("Service reads speeches")	CRIT*	#64, #63, #23 · prod already patched
SSRF in <code>brand/extract</code> (fetches user URL, reflects content)	HIGH	#66, #68, #39
SSRF in <code>seo</code> route (DNS-rebind / redirect bypass)	HIGH	#68, #62, #56, #59
SSRF in <code>gigs/calendar</code> (stored, recurring)	MED	#68, #56
Editor storage IDOR (client path → admin client)	MED	#63
Missing rate limits on cost-driving AI routes	LOW	#63, #48, #30
Next.js dependency CVE patch	MED	#66, #63

*Critical in the repo's `schema.sql` for fresh installs; production was separately verified to already lack this policy. Merging any one of #64/#63 fixes the source file.

The one new finding — billing trigger

NEW The billing-column protection trigger never recognizes the service role, so it fails closed.

`supabase/subscription.sql` guards `profiles.plan / stripe_*` so a user can't self-upgrade from the browser. The guard tests `current_user <> 'service_role'` — but the function is `SECURITY DEFINER`, and inside such a function `current_user` is the function *owner* (e.g. `postgres`), never the calling session. So the check is essentially always true.

Effect: it blocks the browser self-upgrade (the intended part) but *also* blocks the legitimate Stripe-webhook and admin plan update — the service-role client reaches Postgres as role `service_role` via PostgREST, which the trigger can't see. Paid subscriptions would silently fail to activate. It hasn't bitten yet only because paywalls are off (`PAYWALLS_DEFAULT = false`), so no real plan change has been attempted.

Fix (in the migration): check the JWT role claim — `current_setting('request.jwt.claims', true)::jsonb ->> 'role'` — which PostgREST sets per request and `SECURITY DEFINER` doesn't rewrite. Verify against prod before enabling paywalls: a Stripe test checkout should set `plan`, and a browser attempt to change it should still raise.

Findings not yet covered by a PR (lower severity)

MED Non-functional PKCE on Twitter/X & TikTok OAuth

`lib/clipflow/platforms.ts` hardcodes `code_challenge: 'challenge'` with method `plain` and sends the same constant as the verifier. That's PKCE in name only. Residual risk is limited — these are confidential server-side clients with a real `client_secret`, and the `state` cookie still blocks CSRF — but the PKCE

layer does nothing. Fix: generate a random verifier per authorize call, store it in an httpOnly cookie, send `S256(verifier)` as the challenge.

LOW Encryption-at-rest hardening

Two schemes coexist: ClipFlow tokens use `scryptSync` with a *static* salt and fall back to reusing `SUPABASE_SERVICE_ROLE_KEY` as key material; agent tokens use a bare `sha256(APP_ENCRYPTION_KEY)`. Both are AES-256-GCM with a random IV, so neither is exploitable on its own — but consolidating on one dedicated `APP_ENCRYPTION_KEY` with a per-value random salt would be cleaner.

LOW User enumeration & raw error messages

`auth/signup` returns a distinct 409 for an existing email (contrast with `auth/resend`, which deliberately doesn't). Several 500s return raw `err.message`, and the Google OAuth callback reflects the token-exchange error into the redirect URL — minor internal info disclosure.

Verified clean (no action needed)

- No secrets in any tracked file; `.env.local` / `.env` are gitignored, only the placeholder example is committed.
- Every API route authenticates; every `[id]` read/write is scoped by `.eq('user_id', ...)` — including where the service-role client is used. No classic DB IDOR, no missing-auth route.
- Admin routes gate on `ADMIN_EMAIL`; `admin/action` uses HMAC-signed, expiring, constant-time-verified tokens and escapes user HTML.
- Stripe webhook verifies the signature and is the only path that changes a plan; donations are anonymous and can't alter plans.
- OAuth `state` CSRF protection present and validated on both flows; third-party tokens encrypted before storage, never returned to the browser (only a 4-char hint).
- Open-redirect guard on the auth callback; security headers (HSTS, X-Frame-Options, nosniff, Referrer-Policy, Permissions-Policy) set globally.
- All 28 tables have RLS enabled; encrypted-secret tables only ever return ciphertext to the owner.

Dashboard / config items (can't be done in code)

- **Supabase → Auth → Passwords:** enable "Prevent use of leaked passwords" — the one remaining security-advisor warning (from your own checklist).
- **Verify RLS on ad-hoc tables:** `reset_tester_account` references tables not in version control (`clipflow_uploadpost_keys`, `editor_projects`, `script_projects`, `timeline_projects`, `aeo_tips`, `aeo_settings`). A few (`keynotes`, `keynote_variants`, `user_tips`) are also read by anon-client routes that rely on RLS alone. Confirm each has an owner-scoped `user_id` policy — the code has no second line of defense there.
- **Supabase MCP wasn't authorized** this session, so live policies couldn't be inspected directly — findings above are from the SQL files and code. Authorize it to let a future run verify prod directly.

Recommendation

1. **Triage the PR backlog first.** Pick one canonical hardening PR (#63 and #66 are the most complete and recent), merge it, and close the older redundant ones (#23, #30, #32, #39, #48, #56, #59, #62). That single action resolves almost every finding in this report.
2. **Run `supabase/hardening_2026_07.sql`** once in the Supabase SQL editor — it closes the SQL layer including the new billing-trigger bug.
3. **Before switching paywalls on**, test a Stripe checkout end-to-end to confirm the billing trigger now lets the webhook set `plan`.
4. **Reconsider the cadence of this routine.** Re-auditing a codebase whose fixes are already written, unmerged, doesn't add safety — it adds queue. Point it at merging and verifying instead of re-discovering.

Note on the run's brief: I stopped at genuinely useful work rather than looping to burn the token budget or making broad unattended edits to a live site. A pile of speculative changes would have made the backlog problem worse, not better.

Generated automatically as a background routine. Findings are from static review of the repository and are a security baseline, not a formal penetration test or legal advice — have the billing-trigger and PKCE items verified against production before relying on them.